

Stock Trading Optimization by Deep Reinforcement Learning

Muhammed Gölçük
EEE Dept.
Bilkent University
Ankara, Turkey
muhammed.golcuk@ug.bilkent.edu.tr

İbrahim Orcan Ön
EEE Dept.
Bilkent University
Ankara, Turkey
orcan.on@ug.bilkent.edu.tr

Ertuğrul Çoban
EEE Dept.
Bilkent University
Ankara, Turkey
ertugrul.coban@ug.bilkent.edu.tr

Dağhan Ünal
CS Dept.
Bilkent University
Ankara, Turkey
daghan.unal@ug.bilkent.edu.tr

Gökтуğ Çağırın
CS Dept.
Bilkent University
Ankara, Turkey
goktug.cagiran@ug.bilkent.edu.tr

Abstract—This project explores the use of Deep Reinforcement Learning (DRL) in stock market investment, focusing on the Double Deep Q-Network (DDQN), Proximal Policy Optimization (PPO), and Asynchronous Advantage Actor-Critic (A3C) models. Utilizing data from Borsa Istanbul (2015-2021), the study aims to enhance trading strategies through these advanced algorithms. The project explores the potential of DDQN, PPO, and A3C in handling the complexities and dynamic nature of financial markets, with an emphasis on managing exploration-exploitation balance and adapting to non-stationary environments. The primary evaluation metric for the study is cumulative return. This project highlights the potential of DRL in optimizing financial decision-making in complex market conditions.

Index Terms—Finance, Deep Reinforcement Learning, PPO, Double DQN, A3C

I. INTRODUCTION

This final report elucidates the outcomes of our project, which is centered on the application of advanced Deep Reinforcement Learning (DRL) methodologies, specifically the Double Deep Q-Network (Double DQN), Proximal Policy Optimization (PPO), and Asynchronous Advantage Actor-Critic (A3C) algorithms, within the intricate and ever-evolving domain of financial investment. The primary objective of our research is to harness the potential of DRL to streamline and enhance decision-making processes pertinent to both individual and institutional investors in the realm of finance.

The core of our project involves training a DRL agent to proficiently execute investment decisions, namely buying, holding, or selling, within an action space defined as $(-1, 0, 1)$. These decisions are grounded in the analysis of historical stock value data, particularly from Borsa Istanbul [1]. To facilitate this endeavor, we employ the OpenAI gymnasium and gym-trading environment [2]. This choice of environment underscores our commitment to integrating robust and realistic financial data representation into our project.

A notable aspect of our approach is the self-implementation of the aforementioned algorithms using PyTorch, a leading

library in the machine learning community. This decision was driven by the desire to have granular control and flexibility over our algorithmic implementations, allowing us to tailor them specifically to the nuances of financial data and decision-making.

The ambition of this project extends beyond mere academic interest; we aim to unearth a viable, optimized strategy for navigating the stock market, a landscape characterized by its complexity and unpredictability. By leveraging the computational prowess of DRL algorithms, we aspire to provide a tool that can significantly augment the decision-making capabilities of investors in this challenging environment.

This report delves into the literature review of algorithms and similar projects, the dataset used, the methodologies implemented, and the results and analysis of our project.

II. LITERATURE REVIEW

The literature review for the project encompasses several pivotal studies and resources, each contributing uniquely to the field. The FinRL library [3], documented thoroughly in its own manual, stands out as a comprehensive framework designed for training and evaluating Reinforcement Learning (RL) agents in financial contexts. Its significance lies in its ability to integrate realistic financial datasets and advanced Deep Reinforcement Learning (DRL) methodologies.

Complementing this is the Borsa Istanbul Turkish Stock Exchange dataset by G. Kesler [1], a rich resource available on Kaggle, which offers extensive financial data crucial for understanding the Turkish stock market's dynamics. This dataset is instrumental in training DRL agents, providing valuable insights into market fluctuations and economic events through its detailed historical stock value data.

The Gym Trading Environment [2], documented in its online resources, provides a specialized platform for simulating stock trading scenarios. Its sophisticated plotting and visualization capabilities are essential for analyzing and refining the

trading strategies developed by RL agents, making it a vital tool for practical evaluations. Further, the AI4Finance Foundation’s FinRL repository on GitHub extends the capabilities of the FinRL library. It offers advanced tools for multiple stock trading and portfolio management simulations, highlighting the need for robust environments to test the scalability and adaptability of DRL agents in diverse market conditions[4].

Additionally, the studies by Z. T. Wang and M. Ueda [5], H. van Hasselt, A. Guez, D. Silver [6], J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov [7], V Mnih. et al. [8]., and X.-Y. Liu et al. [9], provide critical insights into the development and refinement of DRL models, which we use. These studies focus on convergent and efficient deep Q-network algorithms, double Q-learning to reduce overestimation bias, Proximal Policy Optimization (PPO) algorithms for stable training, and we use these practical approaches to DRL in stock trading. After this literature review, we implemented these algorithms from these papers from scratch with Pytorch.

Collectively, these resources form a comprehensive foundation for the study, blending theoretical insights, practical tools, and rich datasets to advance the application of DRL in optimizing stock trading strategies.

III. DATASETS AND ENVIRONMENTS

A. Dataset

In our study, we utilized the comprehensive Borsa Istanbul (BIST) dataset [1], a rich and expansive collection of financial data that encompasses nearly all the tradable stocks on the BIST. Spanning from December 2015 to the close of 2021, this dataset offers a unique window into six years of the Turkish stock market’s dynamics. Each entry in the dataset is timestamped with a daily frequency, providing an in-depth, day-to-day chronicle of market fluctuations, trends, and patterns.

This period was marked by a range of significant global and local economic events, making the dataset not only vast in its coverage but also diverse in the financial narratives it encapsulates. From geopolitical shifts to domestic economic policies, each factor played a role in shaping the market trends captured in this dataset. This will be challenging for algorithms since we don’t provide real-world information about the changing dynamics of the world or social media influence.

Furthermore, the granularity of daily updates presents an invaluable resource for analyzing short-term market movements alongside longer-term trends. Such detailed data allows for a myriad of analyses, from predictive modeling and trend analysis to more complex, AI-driven market simulations.

Moreover, the BIST dataset stands out due to its inclusion of a wide array of stocks, representing various sectors and company sizes. This diversity offers a comprehensive overview of the market, making it an ideal resource for studies in financial economics, portfolio management strategies, and even behavioral finance research.

In essence, the BIST dataset is not just a collection of numbers and dates; it’s a narrative of Turkey’s financial heartbeat

over a substantial and tumultuous period, offering a treasure trove of insights for analysts, researchers, and economists alike.

B. Data Cleaning and Preprocessing

In our research, a critical step was to identify and refine the necessary data points to effectively implement our environment and train our agents. The initial dataset we accessed was comprehensive, containing a wealth of information. However, much of this data fell outside the scope of our study. For instance, details like short positions and trade report counts, while valuable in certain contexts, were not pertinent to our specific research objectives. Therefore, our first task was to meticulously clean the dataset to retain only the most relevant fields:

- **Date:** The date on which the trades occurred. This temporal marker is crucial for tracking stock performance over time.
- **Open:** The opening price of the stock on the given date. This is a key indicator of the stock’s initial market position for the day.
- **Low:** The lowest price at which the stock traded on that date. This metric helps in understanding the volatility and the downside risk for the day.
- **High:** The highest price at which the stock traded on that date. It provides insight into the upward potential and market optimism.
- **Close:** The closing price of the stock on that date, which is critical for assessing the day’s overall performance.
- **Tic:** The stock’s symbol or Ticker Symbol, essential for identifying and tracking individual stocks.

After refining the dataset to include only these fields, we proceeded to remove stocks with sparse data. Specifically, we excluded those stocks that had fewer than 300 entries to ensure a sufficient depth of data for meaningful analysis. During this phase, our thorough investigation revealed the presence of erroneous data entries, such as multiple entries for a stock on the same date. We meticulously removed these anomalies to ensure the integrity and reliability of our dataset.

With the dataset now cleaned and streamlined, we enhanced it by incorporating technical indicators for each entry. These included indicators like MACD (Moving Average Convergence Divergence), RSI 30 (30-day Relative Strength Index), and DX 30 (30-day Directional Movement Index). The addition of these technical indicators was a strategic decision, aimed at providing deeper insights into market trends and stock performance metrics that are vital for effective trading strategies.

Another significant step in our data processing was the calculation of covariance matrices for each stock entry. Recognizing the importance of risk assessment in stock trading, we used a one-year (252 trading days) lookback period for these calculations. This approach not only aligned with standard financial analysis practices but also gave us a robust framework for understanding the relationships and risk dynamics between different stocks. However, to maintain the focus and relevance

of our dataset, the first 252 entries for each stock—used primarily for these covariance calculation were subsequently removed.

As a result of this comprehensive data processing, the earliest entries in our final, preprocessed dataset date back to December 2016. This dataset, with its focus on relevance and accuracy, provided a solid foundation for training our RL agents, ensuring that they learned and adapted from data that was both pertinent and reliable.

C. Environment

In our project, we employed the sophisticated OpenAI Gymnasium, a versatile platform known for its robustness and flexibility in training and testing reinforcement learning (RL) agents. OpenAI Gymnasium stands out in the realm of AI research for its wide array of environments, each tailored to different aspects of machine learning and artificial intelligence. Among these, several third-party environments are specifically designed for simulating stock markets and training RL agents in trading scenarios. After thorough consideration, we selected two distinct environments for our experiments: the Gym Trading Environment and FinRL, catering to different aspects of our research in trading [2]–[4].

The Gym Trading Environment was our choice for experiments involving single-stock trading. One of the standout features of this environment is its sophisticated plotting and visualization capabilities. These tools are invaluable as they allow for a clearer understanding and interpretation of the trading strategies developed by the RL agents. The visual representations provided by the Gym Trading Environment offer insights into our agents' decision-making processes, making it easier to analyze and refine their trading strategies. Moreover, the environment's fidelity in simulating real-world single-stock trading scenarios provides a reliable testbed for our agents.

On the other hand, FinRL was selected for its exemplary performance in simulations involving multiple stock trading and managing portfolios. The environment is highly regarded for its comprehensive and realistic simulation of complex market dynamics. It enables our agents to engage in more sophisticated trading strategies that involve a variety of stock and portfolio management tactics. FinRL's ability to mimic real-world market conditions across a range of stocks and portfolios is particularly beneficial for testing the scalability and adaptability of our trading agents [4]. This environment also offers tools for risk management and performance evaluation, which are critical for understanding the effectiveness of our RL agents in diverse market scenarios.

Both these environments, with their unique features and capabilities, have been instrumental in our research. The Gym Trading Environment allows for an in-depth analysis of single stock trading strategies, while FinRL provides a broader canvas to test and refine portfolio management tactics.

In the Gym Trading Environment, we also implemented a preprocessing function that randomizes data. For the training data,, it selects a random starting date in the first 100 days of

the data and a random ending date between the last 450 and 400 days. For testing, it selects a random starting date between the last 400 and 350 days.

Overall, in the end, we used gym-trading-env since we wanted to demonstrate single-stock trading, but we implemented both for experimenting. The following results of this report were created with the Gym Trading Environment.

IV. DRL MODELS & METHODOLOGY

In our endeavor to address the complexities of financial investment decision-making, we have identified three sophisticated Deep Reinforcement Learning (DRL) models as the focal point of our methodology: Double Deep Q-Network (DDQN), Proximal Policy Optimization (PPO), and Asynchronous Advantage Actor-Critic (A3C). Each model has been meticulously selected and analyzed in the context of our specific problem domain, considering their unique strengths and the challenges they present when applied to stock market data.

As of this final report, we have successfully implemented and tested all models.

A. Double DQN (DDQN)

The DDQN model emerges as an advancement over the conventional Deep Q-Network (DQN), aiming to mitigate the inherent overestimation bias in the estimation of Q-values found in DQN. This refinement leads to a more stable learning process, which is crucial for applications in environments with high volatility, such as financial markets [5], [6].

1) *Comparison to DQN*: In the standard DQN framework, the Q-learning update rule is given by [5]:

$$Q_{\text{new}}(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (1)$$

Here, α represents the learning rate, γ is the discount factor, s and s' are the current and next states, respectively, a is the action, and R is the reward. The term $\max_{a'} Q(s', a')$ introduces a positive bias, as it uses the same network to both select and evaluate an action. DDQN modifies this approach by employing two networks: a current network for action selection and a target network for evaluating the chosen action. The Q-value update in DDQN is [6]:

$$Q_{\text{new}}(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{current}}(s', a')) - Q(s, a) \right] \quad (2)$$

The key difference is in the term $Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{current}}(s', a'))$. Here, the action a' that maximizes the Q-value for the next state s' is selected using the current network, but the Q-value for this action is obtained using the target network. This separation effectively reduces the overestimation bias.

2) Challenges:

• Non-stationarity of Financial Markets:

DRL algorithms typically assume a stationary environment. However, financial time series data are inherently non-stationary, characterized by changing statistical properties over time. This non-stationarity poses a significant challenge for the DDQN model.

• Exploration-Exploitation Dilemma:

Exploration-Exploitation Dilemma: As training progresses, the balance between exploration (trying new actions) and exploitation (leveraging known information) becomes increasingly vital. In the context of financial markets, this balance is crucial to adapt to market changes while leveraging learned strategies.

• Replay Memory Inefficiency:

DDQN, like DQN, utilizes a replay memory to store and sample experiences. However, uniform sampling from this buffer may not be the most efficient method. This is because it treats all experiences equally, whereas some experiences might be more valuable for learning than others. Implementing strategies like prioritized experience replay could potentially address this inefficiency.

3) **Double DQN Implementation:** In this project, we have adopted PyTorch as the primary library to implement our reinforcement learning models. The coding environment is structured to accommodate various functions and classes that are pivotal for the Double DQN algorithm. Our approach was to first implement DQN and call their functions in a separate class for Double DQN and initiate two separate deep q networks from DQN classes and functions.

For exploration, we implemented several functions in the intermediate report, such as epsilon greedy and uniform random policy. However, as of the final report, we removed the epsilon greedy and uniform random after discovering that they weren't contributing much to an exploration. Look for the intermediate report for the explanation. Furthermore, to refine the code, we removed separate DQN classes and made one DDQN class.

B. Proximal Policy Optimization (PPO)

PPO is a deep reinforcement learning method designed to maintain stable training by ensuring policy updates are not too drastic [7]. It prevents large policy updates that could lead to instability thanks to its loss function and gradient clipping during the optimization (discussed further in the next sections).

PPO belongs to the family of policy gradient methods, and it operates by iteratively collecting data through interaction with the environment and then updating the policy based on the collected data.

1) Implementation of the PPO Algorithm:

• Actor and Critic Networks:

The *actor-network* is responsible for representing the policy. It takes the current state as input and outputs a probability distribution over possible actions. In our implementation, the actor-network has three fully connected layers with ReLU

activations, and the final layer uses softmax to generate action probabilities.

Similarly, the *critic-network* is responsible for estimating the value function, representing the expected cumulative reward for a given state. This value is used to estimate the advantage in the policy update. In our implementation, the critic network also has three fully connected layers with ReLU activations, and it outputs a single value which is the estimate of the cumulative reward for a given state. In both networks, Adam optimizer is chosen in both networks for weight updates [7].

• Memory:

A Memory class is created to store the experiences (states, actions, probabilities, values, and rewards) collected during interactions with the environment. This memory is then sampled to generate batches for training the actor and critic networks.

• Agent:

An agent class is created as a core component to coordinate the training process, manage the neural networks (actor and critic), and interact with the environment through the "Memory" class to collect experiences.

The agent collects experiences by choosing an action (from the action space Buy, Sell, Hold) based on the current policy and it stores them in memory. It performs multiple epochs training to collect data and then updates the networks based on the mini-batches stored in the memory. Firstly, the advantage function is calculated using the Generalized Advantage Estimation (GAE) method [7]. It is basically the difference between the actual return and the estimated return. Then, the actor loss is calculated based on the following formula:

$$L^A(\theta) = -\mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] \quad (3)$$

where $r_t(\theta)$ is the probability ratio of the new policy to the old policy.

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (4)$$

This term allows us to avoid big policy updates as well as gradient clipping. The epsilon is a small clipping parameter to prevent exploding gradients.

Critic Loss is the mean squared error between the estimated and actual return, calculated by the formula:

$$L^C(\theta) = \mathbb{E}_t [(G_t - V_\theta(s_t))^2] \quad (5)$$

Our algorithm is trying to minimize the total loss which is the combination of actor loss and critic loss.

$$L^{PPO}(\theta) = L^A(\theta) + \frac{1}{2}L^C(\theta) \quad (6)$$

In addition to these in the early episodes of training an entropy term is added to the actor loss to increase the exploration. Entropy is a measure of uncertainty or randomness in a probability distribution of the actions that will be selected. The coefficient of the entropy is a hyperparameter and it is

selected as 0.001. It can be assigned a higher value if the aim is to increase the exploration and discourage the policy from converging to a highly deterministic one.

In the training process, the model is saved in the episodes in which it gives the best score (highest portfolio value it returned).

2) **Challenges:** Conservative updates (the algorithm can sometimes be too cautious, leading to slower convergence). Hyperparameter sensitivity, on-policy data need (PPO demands new environment samples for every update, which may be inefficient). non-stationarity environment (a financial market environment that changes rapidly can struggle the algorithm), exploration limitations. Conservative Updates: PPO's cautious approach in updating policies to avoid instability can lead to slower convergence rates as the algorithm does not exploit the learning opportunities as well.

Hyperparameter Sensitivity: PPO's performance depends on the choice of hyperparameters, that means the challenge of choosing the right balance in settings like learning rate and discount factor is very important for better performance.

On-Policy Data Requirement: PPO being an on-policy algorithm means that it needs new data samples from the environment after each policy update. This can lead to inefficiencies since providing the samples means to have continuous interaction with the environment. This interaction is resource-intensive

Changing Financial Markets: Financial markets are rapidly changing environments. In such an environment, PPO may struggle since the algorithm's assumption is that the environment's dynamics remain relatively stable between the policy updates. Thus, reducing the effectiveness of the model.

C. Asynchronous Advantage Actor-Critic (A3C)

The Asynchronous Advanced Actor-Critic (A3C) algorithm represents a significant leap in Deep Reinforcement Learning, combining both policy-based and value-based approaches.

It functions on the principle of parallelism, where each agent (comprising an actor-critic pair) operates in its unique instance of the environment and asynchronously updates a global model. The core methodology of A3C is grounded in its distinct policy gradient formula:

$$\nabla_{\theta'} \log \pi(a_t|s_t; \theta') A(s_t, a_t; \theta, \theta_v)$$

In this equation, $\pi(a_t|s_t; \theta')$ denotes the policy network that outputs the probability of action a_t given state s_t and is parameterized by θ' . The term $A(s_t, a_t; \theta, \theta_v)$ is the advantage function, indicating how much better an action is compared to the average.

It is computed by subtracting the value function $V(s_t; \theta_v)$ from the cumulative reward R_t summed with the value of the subsequent state $V(s_{t+k}; \theta_v)$, discounted by γ to the power of k . The advantage function provides an essential signal for the policy gradient, driving the updates to favor actions that yield higher returns than the current policy's average.

The value function $V(s_t; \theta_v)$ estimates the expected cumulative reward from state s_t onwards, serving as a critic that

assesses the policy's decisions. The advantage function is thus an estimate given by:

$$A(s_t, a_t; \theta, \theta_v) = \sum_{i=0}^{k-1} \gamma^i r_{t+i} + \gamma^k V(s_{t+k}; \theta_v) - V(s_t; \theta_v)$$

This sum accumulates rewards r_{t+i} over a lookahead window of k steps, discounted accordingly, and contrasts it with the current value estimate.

The asynchronous update of the A3C algorithm is pivotal for exploring diverse policies and stabilizing training in complex environments, such as stock trading. The asynchrony allows for multiple agents to learn concurrently, each potentially exploring different strategies, which enhances the robustness and generalizability of the learned policy.

1) **Challenges:** Implementing the A3C methodology entails overcoming several significant challenges:

Hyperparameter Sensitivity: A3C's performance is highly contingent on the proper selection of hyperparameters, such as learning rates and discount factors, which must be finely tuned for effective convergence and optimal performance.

Non-Stationarity of Financial Markets: Financial markets are dynamic, with rapidly changing conditions. The A3C algorithm must continuously adapt to these changes to maintain stability in the learning process and consistency in decision-making.

Balancing Actor and Critic: Finding the right balance between the policy (actor) and the value function (critic) is challenging but essential since misalignments in their learning rates or objectives may be suboptimal for the model's training.

Complexity Due to Asynchronous Updates: Multiple agents updating a global model asynchronously necessitates intricate synchronization mechanisms to ensure coherent learning and decision-making.

Sensitivity to Noise: The A3C algorithm's sensitivity to noise can impact the stability of learning and the quality of the policy developed.

2) **A3C Implementation:** In the implementation of A3C, eight asynchronous workers with local actor-critic networks and optimizers, a global actor-critic network, and a global optimizer were utilized.

V. RESULTS & ANALYSIS

Our comprehensive evaluation of the deployed Deep Reinforcement Learning (DRL) models centers around the metric of cumulative return, which serves as the cornerstone for validation. This metric is pivotal in measuring the profitability and success of our trading strategies over time. In our analysis, we have taken an additional step to plot the averages of the algorithms, providing a clearer visual representation of their performance across various runs.

Moreover, for a more illustrative demonstration of the models' capabilities, we have captured plots from the trading environment that detail the performance of specific cases. These snapshots offer a tangible glimpse into the decision-making processes of the algorithms and their real-time interactions with the market.

Furthermore, by the time of this final report, we had implemented random initialization of both tests and training data to test our models, which we received as feedback.

A. Double DQN Results

In our comprehensive evaluation of the Double DQN model, the initial investment was set at 1000 Turkish Lira (TL). The agent, operating within this DRL framework, was tasked with executing a range of investment actions: buying, selling, or holding, using the full extent of the portfolio for each transaction. The training regimen for the agent was strategically capped at 100 episodes for each stock data, integrated with a self-stopping algorithm activated upon achieving stabilization or diminishing returns in cumulative rewards. The entire training duration was recorded, amounting to 52 minutes for each model.

During the training, we encountered notable exploration challenges, particularly when comparing the Double DQN model's performance with that of the PPO algorithm. A recurring pattern was observed in the Double DQN model, where the same actions of buying and selling were repetitively executed across various epochs (episodes). To address this, we diversified our target policies beyond the standard greedy policy, incorporating the epsilon greedy policy and uniform random policy. These policies were primarily implemented to enhance exploration and to assess the learning differences. Given that DoubleDQN is an off-policy algorithm, balancing exploration with exploitation is critical, especially since the experience replay buffer leverages past experiences to inform current decisions. However, after training for several epochs, we understood that even though we used different target policies, it converged to an eventuality.

To tackle the non-stationarity of financial data, we incorporated additional metrics, such as the volume of trades, RSI, MACD metrics into the inputs of our neural network (NN). This inclusion allowed the NN to discern correlations, thus providing valuable insights to the RL algorithm about the underlying market dynamics.

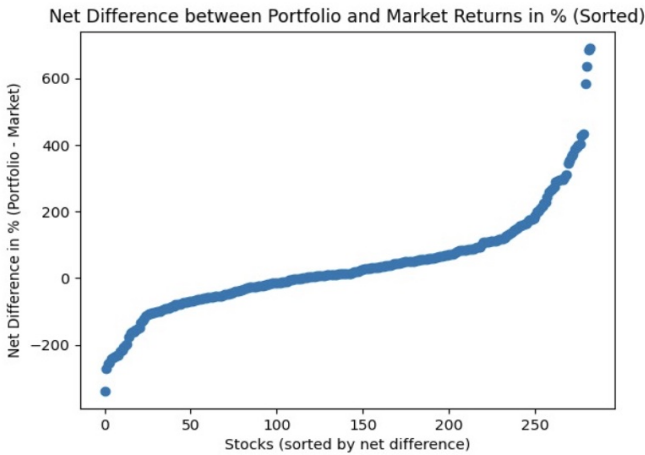


Fig. 1. DDQN - Net Difference between Portfolio and Market Returns

For performance evaluation, the agent was trained on different stock datasets for 100 episodes and subjected to real-time simulation. The number of episodes is decided by trials, and it is observed that when the number of episodes is less than 100, it does not fully learn the pattern in the data and results in underfitting. On the other hand, for higher episode values, the model has overfitted on training data, and unrealistic returns are obtained while its performance on the test set is poor.

The agent is subjected to real-time simulation on test data using all stock data, which includes two years of history with randomization. For each stock, the initial investment was set to 1000 Turkish Lira (TL), and the agent was simulated separately. In the figure below, the net difference between portfolio return and market return for each stock can be seen.

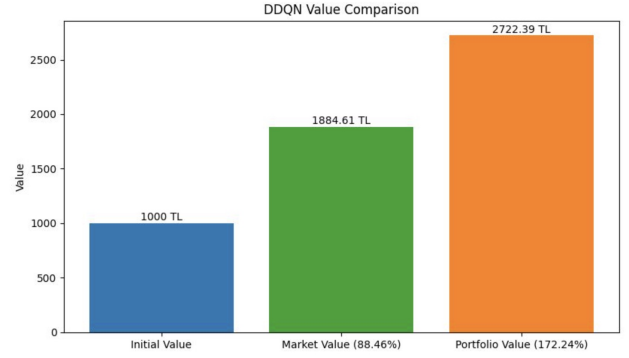


Fig. 2. DDQN Market and Portfolio Value Comparison on Average

To assess the average performance of the model, average market return and portfolio return are also calculated, and the agent obtained %204 average return in the environment where the average market return is %111.

Some randomly chosen stocks are also simulated to see single-model performance. In Figure 2, the model tested for KRON stock and obtained %268.86 portfolio return between 2020-01-29 and 2021-12-06, where the market return is %103.48.

Another test involved training the agent with the same configurations on VKING stock data spanning the same period. In this scenario, the agent could not outperform the market and obtained %-26.57 portfolio return where the market return is %56.74.

Overall, the DDQN agent is able to make a profit on average, considering its performance on different stock data in a highly volatile, non-stationary market.

B. PPO Results

For the Proximal Policy Optimization (PPO) model, the initial portfolio was also set at 1000 TL for consistency and simplicity. The training was conducted over 100 episodes using data from Borsa Istanbul, lasting approximately 2 hours. We chose 100 episodes of training to obtain a reasonable amount of training time since we need several training attempts



Fig. 3. DDQN Test on Kront Stock Data



Fig. 4. DDQN Test on VKING Stock Data

to get an insight into model behavior. During training, challenges were encountered with data formatting and algorithmic setbacks, necessitating reruns of the training process. Also, training for longer episodes might lead to overfitting on the training data, which is not desirable since we focus on a specific time period during training.

One notable aspect of PPO is its on-policy nature, which inherently prioritizes exploration. This approach led to initial failures and portfolio losses, as the algorithm consistently explores new strategies without leveraging past experiences. Additionally, the deterministic nature of on-policy algorithms

resulted in repetitive actions across different runs, sometimes leading to similar losses.

Despite these challenges, the PPO model demonstrated its ability to learn and adapt over time. The cumulative return, calculated as the difference in portfolio value before and after each action, provided a measure of the total profit generated by the agent over a 5-year period (December 2015 - January 2020) in the training process using Borsa Istanbul data. Model test is done on the data over a 1-year 11-month period (January 2020 - December 2021).

The average run of our PPO agent resulted in a final

portfolio value of 2013.66 TL. Although the model brings a profit, it has failed to exceed market return. This outcome, alongside comparative analyses of market returns and portfolio returns, is graphically represented in the accompanying figures, illustrating the model's performance and effectiveness in financial decision-making. In the positions graph, the actions of the agents are represented. Position 1 means the agent buys as many stocks as possible with its portfolio, and position 0 means it sells all the stocks it has.

The following are some underlying reasons that might have caused the mentioned poor performance by PPO:

- **Exploration-Exploitation Balance:**

PPO, functioning as an on-policy algorithm, places a significant emphasis on exploring new strategies, even if they result in initial losses. This characteristic contrasts with off-policy algorithms that can more effectively leverage past experiences for exploitation. This tendency to prioritize exploration may pose challenges, particularly when compared to off-policy counterparts known for efficiently exploiting historical data.

- **Market Dynamics and Non-Stationarity:**

The financial markets exhibit dynamic and non-stationary behavior, indicating continuous changes over time. In the case of PPO, being trained on a 5-year historical dataset, there's a potential mismatch with the current market dynamics. This mismatch can impede the model's ability to adapt swiftly and make optimal decisions in response to the evolving nature of financial markets.

- **Risk-Averse Behavior of PPO:**

While the proximal policy constraint contributes to training stability, it introduces a risk-averse aspect to the model. This risk aversion, while enhancing stability, may also lead to a slower response to sudden shifts or emerging profitable trends in financial markets. Consequently, the model's cautious approach might result in delayed capitalization on new opportunities.

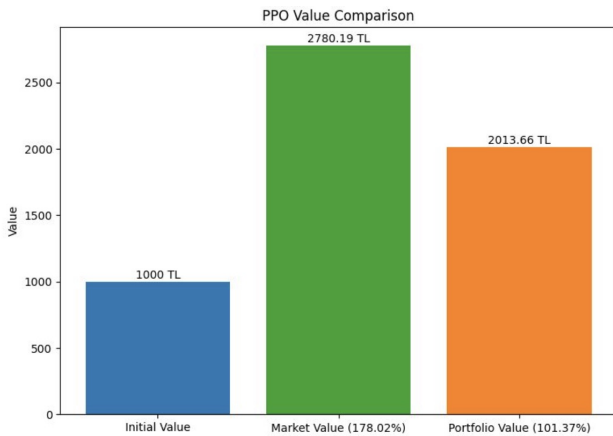


Fig. 5. PPO Market and Portfolio Value Comparison on Average

C. A3C Results

The evaluation of the Asynchronous Advantage Actor-Critic (A3C) model revealed some distinctive features and outcomes.

The training regime was characterized by its speed, with each agent completing about 3000 episodes in an approximate duration of 6 minutes. This efficiency can be attributed to the implementation of 8 worker cores, which allowed parallel processing and significantly reduced training time. However, the A3C model demonstrated a high degree of variability in performance between different runs. This inconsistency in results points to a lack of stability and predictability, which are crucial attributes in the context of financial decision-making. A critical observation from the trials was the A3C model's performance in comparison to market returns. It was noted that, on average, the model failed to exceed market returns in 94% of the episodes. This consistency in performance raises questions about the model's effectiveness in a financial market context. The analysis suggests several contributing factors to these results:

- **Predominance of Exploration in On-Policy Algorithms:**

The A3C model's intrinsic emphasis on exploration often comes at the expense of exploiting known, successful strategies. This characteristic, while beneficial for discovering new strategies, may limit the model's ability to capitalize on existing market trends.

- **Importance of Long-Term Trends in Financial Analysis:**

The focus on short-term outcomes, as opposed to long-term market trends, likely impacted the model's performance. Financial markets are often driven by longer-term trends, and the A3C model's approach may not have aligned well with this aspect.

- **Multi-Agents with Short Memories Operating on Varied Datasets:**

The A3C's use of multiple agents, each with a limited memory span of 5 steps and operating on different datasets, could have hindered the model's ability to learn and adapt to more extended market patterns and trends.

- **Impact of Worker Cores on Accuracy and Episode Implementation Time:**

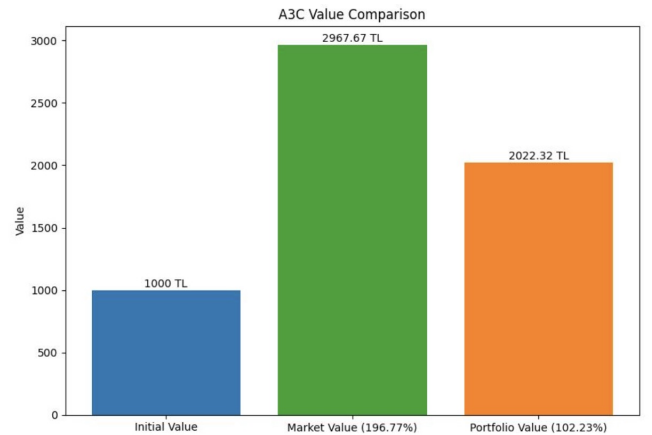


Fig. 7. A3C Market and Portfolio Value Comparison on Average



Fig. 6. PPO Test on DAGHL Stock Data

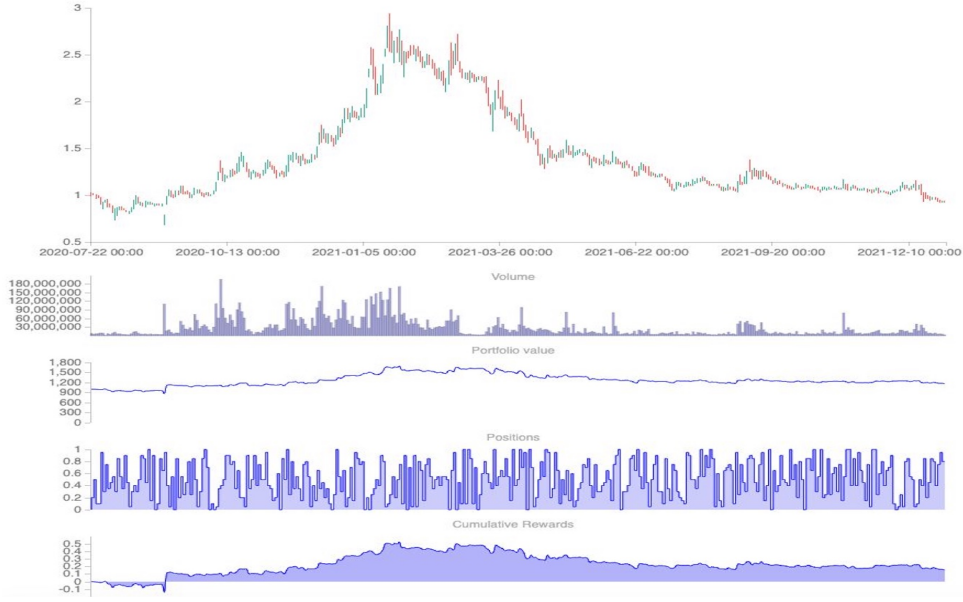


Fig. 8. A3C Test on YYAPI Stock Data

The model's configuration with eight worker cores played a pivotal role in its operational dynamics. It was observed that increasing the count of worker cores led to improvements in accuracy and a reduction in the time taken for episode implementation. This scalability suggests that the model's performance can be enhanced by increasing parallel processing capabilities. However, this also indicates a trade-off between computational resources and performance gains. In conclusion, while the A3C model exhibited high efficiency in training speed, its considerable variability and general inability to outperform market returns in a majority of episodes indicate

areas for potential improvement. The results emphasize the need for a more balanced exploration-exploitation strategy, a deeper focus on long-term market trends, and an evaluation of the memory span in multi-agent systems. Additionally, the role of worker cores in enhancing accuracy and reducing episode times presents an avenue for further optimization of the model.

VI. FURTHER DISCUSSIONS

In this section, we critically examine the design choices and algorithmic strategies employed in our study, highlighting the rationale behind selecting specific Deep Reinforcement Learning (DRL) models for the financial environment.

A. Design & Algorithm Choices

The design and algorithmic decisions in this research were driven by the objective of identifying the most effective DRL models tailored to the domain of financial trading. Our choices were informed by a combination of empirical observations and a literature review.

B. DDQN over DQN

The initial phase of our investigation involved the implementation of both the Deep Q-Network (DQN) and the Double Deep Q-Network (DDQN). The standard DQN, while foundational to the field of DRL, presented challenges in the form of overestimation bias. This bias led to an inflated expectation of cumulative rewards, propelling the persistence of suboptimal policies. In the context of stock trading, where precision in prediction is crucial, such overestimation biases could significantly derail investment strategies, leading to losses. To counteract this, the DDQN was introduced, offering a more nuanced approach by decoupling the selection of actions from their evaluation. This bifurcation effectively diminishes the overestimation issue, thereby aligning the model's predictions more closely with actual market performance.

C. Why Choose DDQN, PPO, A3C

The selection of DDQN, PPO, and A3C models was guided by a strategic intent to evaluate and compare different DRL paradigms, specifically, the off-policy versus on-policy approaches and the implications of single-agent versus multiple-agent frameworks. The DDQN was favored in part due to its popularity and proven track record in the literature, suggesting its potential applicability and effectiveness in financial trading scenarios. Conversely, the A3C model's presence in literature, particularly in finance, was limited, which piqued our interest. By implementing A3C, we aimed to fill this gap and explore the potential benefits of its asynchronous, multi-agent approach in the financial domain. The comparison of these models would provide comprehensive insights into their respective strengths and weaknesses in managing the complex, dynamic nature of financial markets. This comparative analysis is crucial for our goal of identifying the most viable DRL strategy for financial decision-making, contributing novel findings to the field.

D. Future Work

This paper implements RL algorithms as they are outlined in their original paper. Further work can be done to fine-tune these models and compare their fine-tuned versions. Aside from the algorithms and models used in this paper, there are also many new RL algorithms, models, and approaches. These new models can be implemented and tested against each other using the same environment and dataset.

VII. CONCLUSION

Our final report details the research, methodology, and findings of our project, which explores the application of Double DQN, PPO, and A3C algorithms in financial investment. The

project's core objective is to utilize the computational prowess of state-of-the-art DRL to enhance decision-making in financial markets, addressing the complexities and uncertainties inherent in stock trading.

Leveraging the Borsa Istanbul dataset, our study provides valuable insights into the dynamic Turkish stock market over a significant period marked by global and local economic events. The data's preprocessing, involving cleaning and incorporation of technical indicators, set a solid foundation for training our DRL agents.

Our implementation of Double DQN, PPO, and A3C models using PyTorch demonstrates a commitment to tailoring algorithms to the specificities of financial data. The DDQN model showed the best results among these algorithms in managing the exploration-exploitation balance and adapting to the non-stationary nature of financial markets. Notably, when applied to real-time simulations with Borsa Istanbul stock data, the DDQN model significantly increased portfolio values, showcasing its potential in practical investment scenarios.

However, the PPO and A3C models, despite the profit they bring with respect to the initial portfolio, both failed to exceed market returns, which might have resulted from the predominance of exploration in on-policy algorithms and the importance of long-term trends in financial analysis. All three models' performances were meticulously evaluated, with cumulative return as the primary metric. Based on our results, we can put these models in order as DDQN > PPO > A3C based on their performances. For further improvements of our task in the future, increasing the complexity of our models and having a more detailed dataset including longer time periods of data might be considered in the future.

REFERENCES

- [1] G. KESLER, "Borsa istanbul: Turkish Stock Exchange Dataset," Kaggle, <https://www.kaggle.com/datasets/gokhankesler/borsa-istanbul-turkish-stock-exchange-dataset/data> (accessed Oct. 23, 2023).
- [2] "Gym Trading Environment," Gym Trading Env, <https://gym-trading-env.readthedocs.io/en/latest/index.html> (accessed Nov. 26, 2023).
- [3] "Welcome to FINRL Library!," Welcome to FinRL Library! - FinRL 0.3.1 documentation, <https://finrl.readthedocs.io/en/latest/> (accessed Oct. 23, 2023).
- [4] AI4Finance-Foundation, "AI4Finance-Foundation/FINRL: FINRL: Financial reinforcement learning.," GitHub, <https://github.com/AI4Finance-Foundation/FinRL> (accessed Nov. 26, 2023).
- [5] Z. T. Wang and M. Ueda, "Convergent and efficient deep Q Network algorithm," arXiv.org, <https://arxiv.org/abs/2106.15419> (accessed Nov. 26, 2023).
- [6] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," arXiv.org, <https://arxiv.org/abs/1509.06461> (accessed Nov. 26, 2023).
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv.org, <https://arxiv.org/abs/1707.06347> (accessed Nov. 26, 2023).
- [8] V. Mnih et al., "Asynchronous methods for deep reinforcement learning," arXiv.org, <https://arxiv.org/abs/1602.01783> (accessed Nov. 26, 2023).
- [9] X.-Y. Liu, Z. Xiong, S. Zhong, H. Yang, and A. Walid, "Practical deep reinforcement learning approach for stock trading," arXiv.org, <https://arxiv.org/abs/1811.07522> (accessed Oct. 23, 2023).